

Stopping Rules for AI Deployment Evaluation: A Dual-Lane Rate-Bounded and Saturation-Aware Method

Richard Heimann
South Carolina, Department of Administration
`richard.heimann@admin.sc.gov`

April 10, 2026

Abstract

Pre-deployment evaluation for language models has an unresolved stopping problem. How much testing is enough to support a transparent and defensible release decision? Existing guidance emphasizes lifecycle test, evaluation, verification and validation (TEVV), deployment-specific evidence, and post-deployment monitoring, but it does not provide a widely adopted quantitative stopping rule for evaluation sufficiency. This paper proposes a dual-lane sequential method. Lane A uses representative testing to bound failure rates within deployment-critical slices via one-sided exact binomial confidence limits. Lane B uses targeted or adversarial testing to assess whether the tail of *novel* severe error mechanisms is saturating, using discovery curves, rolling novelty, and Good-Turing missing-mass estimates. The resulting stopping rule advances a system only when severe-failure risk is bounded below an agreed threshold and the targeted discovery process has flattened enough that additional testing is unlikely to materially change the release decision.

The paper contributes three elements: (i) a formal problem statement for deployment-readiness stopping, (ii) a practical composite rule that joins rate bounds and tail saturation, and (iii) a Monte Carlo study comparing the proposed rule against fixed-budget, recent-novelty, and rate-only baselines. In the safe long-tail simulation, premature stops fell from 25.0% under a rate-only rule and 100% under a fixed budget to 5.0% under the proposed dual rule, at the cost of additional evaluations. In an unsafe long-tail scenario, recent-novelty stopping still stopped 100% of the time, whereas the dual rule refused to stop within budget. A synthetic case study for a policy-oriented retrieval assistant illustrates how the method can be instantiated in practice. The proposed method does not prove safety or correctness. Its purpose is to provide an explicit, reviewable evidence structure for arguing that residual deployment risk has been reduced to an acceptable level for a given release stage.

1 Introduction

Organizations deploying generative AI systems increasingly face a question that is statistical, operational, and governance-related at the same time: *when should testing stop?* Teams may benchmark, red-team, and scenario-test an AI system for weeks, yet still be unable to say why the current evidence is sufficient for release. In practice, many release decisions fall into one of two unsatisfactory modes: testing stops because the schedule says so, or it continues “to death” because no explicit stopping rule exists.

This stopping problem is acute for large language models (LLMs). Their behavior is prompt-sensitive, deployment contexts are heterogeneous, benchmark performance does not transport

cleanly to production, and the tail of failure mechanisms can be long. NIST’s AI Risk Management Framework (AI RMF) and the Generative AI Profile explicitly place TEVV across the deployment lifecycle and stress that deployment decisions must be made in terms of residual risk rather than absolute safety claims [National Institute of Standards and Technology, 2023, 2024]. HELM similarly argues for broad coverage and explicit recognition of incompleteness in language-model evaluation [Liang et al., 2023]. However, these frameworks are intentionally general. They say *what kinds of evidence matter*, but not *how much evidence is enough to stop*.

This paper argues that deployment readiness can be framed as the conjunction of two claims:

- (i) **Rate claim:** in each deployment-critical slice, the post-control severe-failure probability is below an agreed threshold with stated confidence.
- (ii) **Tail claim:** under a targeted search process, the discovery of *new* severe error mechanisms has saturated enough that additional testing is unlikely to materially change the decision.

The first claim addresses frequency. The second addresses novelty in the tail. Together they transform “we tested a lot” into a sequential evidence argument.

2 Related work

2.1 Lifecycle TEVV, deployment specificity, and residual risk

NIST’s AI RMF positions TEVV as a lifecycle activity spanning design, development, deployment, and operation, and it treats *residual risk* as the risk remaining after risk treatment [National Institute of Standards and Technology, 2023]. The NIST Generative AI Profile extends this argument to generative AI, noting that pre-deployment TEVV can be inadequate, nonsystematic, or mismatched to real deployment conditions [National Institute of Standards and Technology, 2024]. This is foundational for the present paper because a stopping rule must be stated in deployment-specific terms. The question is not whether a model is “good” in the abstract, but whether remaining uncertainty is acceptable for a specific use, release stage, and control configuration.

2.2 Holistic evaluation and explicit incompleteness

HELM argues that language-model evaluation must broaden beyond single benchmarks and single metrics, while acknowledging what remains uncovered [Liang et al., 2023]. This observation cuts both ways for stopping. On the one hand, narrow benchmarks are not enough. On the other hand, exhaustive coverage is impossible. A usable stopping rule must therefore operate under explicit incompleteness. That is, it must combine targeted breadth with a principled claim about the residual uncertainty that remains.

2.3 Documentation and assurance arguments

Model cards recast evaluation as documentation intended to support downstream judgment, not merely measurement [Mitchell et al., 2019]. Assurance case traditions go further by insisting that evidence must be connected to an explicit argument about why a system is acceptable for an intended use [Adler et al., 2022, Kelly and Weaver, 2004]. This framing shifts evaluation from accumulating metrics to constructing a justified argument. Evidence is interpreted only in relation

to specific claims, and acceptability is treated as context-dependent rather than intrinsic to the system. Recent work in AI assurance reinforces that testing alone is insufficient; credible assurance requires explicit arguments, layered controls, and traceability from evidence to claims [Bloomfield and Rushby, 2024]. The present paper adopts this perspective. The output is not a set of metrics, but an assurance argument that the system is sufficiently reliable for its intended use. Stopping criteria define when this argument is adequately supported, rather than serving as standalone indicators of quality.

2.4 Rare event inference and stop testing

For rare but consequential failures, raw pass rates are misleading. Clopper and Pearson [1934] provide exact binomial confidence limits; when zero failures are observed, the one-sided 95% upper bound is approximately $3/n$, the well-known “rule of three” emphasized by Hanley and Lippman-Hand [1983]. Software testing and reliability growth offer nearby precedents. Goel-Okumoto and Musa-Okumoto reliability-growth models formalize diminishing returns in defect discovery [Goel and Okumoto, 1979, Musa and Okumoto, 1984], while Dalal and Mallows [1988] directly analyze stop testing as a sequential decision problem. Those works are not directly about LLM deployment, but they show that stopping rules can be formalized rather than left to intuition.

2.5 Discovery saturation and unseen categories

The second half of the stopping problem concerns unseen failure mechanisms. Good [1953] introduced the missing-mass perspective that answers the following: “What proportion of probability mass still belongs to categories not yet seen in the sample?” Chao and Jost [2012] later emphasized comparing samples by *coverage* rather than by raw size. These ideas are attractive for AI deployment because severe error mechanisms behave like a long-tailed category discovery process. The exact ecological assumptions do not hold under adaptive adversarial search, but the underlying intuition remains useful: if a large fraction of observed severe mechanisms are singletons, the tail is probably not yet saturated.

3 Problem formulation

Consider an AI system under a fixed release candidate configuration. Let $h \in H$ index deployment-critical slices, such as common traffic, exception handling, tool-use boundary cases, or long-context conflict resolution. A release decision should not be based on aggregate averages alone; it should be based on the slices whose failures matter most. We separate evaluation into two lanes.

- **Lane A (representative lane):** approximate the deployment distribution within each critical slice and estimate the residual severe-failure rate after controls are in place.
- **Lane B (targeted lane):** intentionally search for new severe mechanisms by stressing the system with adversarial, edge-case, and boundary-condition prompts or episodes.

The two lanes answer different questions. Lane A estimates *how often* failures occurs where the system is expected to operate. Lane B estimates whether the search process is still finding *new kinds* of failures.

3.1 Representative-lane quantities

For slice h , let

$$\begin{aligned} n_h &= \text{number of relevant evaluation opportunities,} \\ x_h &= \text{number of uncaught severe failures,} \\ p_h &= \text{true post-control severe-failure probability,} \\ \tau_h &= \text{accepted threshold for slice } h. \end{aligned}$$

A “relevant opportunity” is not any arbitrary interaction. It is an interaction in which the failure mode in question could actually have occurred. This distinction matters because raw chat counts can substantially dilute the denominator.

3.2 Targeted-lane quantities

Let $z_1, z_2, \dots, z_t$ denote the ordered sequence of severe error *types* observed in the targeted lane. Types are defined at the mechanism level, not the prompt-string level. For example, “fabricated citation to a nonexistent policy source” is a type; ten different prompts that trigger that mechanism are not ten different types.

Define:

$$\begin{aligned} K_t &= |\{z_1, \dots, z_t\}|, \quad \text{cumulative distinct severe types,} \\ f_r(t) &= \text{number of severe types observed exactly } r \text{ times by } t, \\ \Delta K_t(w) &= K_t - K_{t-w}, \quad \text{new severe types in the last } w \text{ targeted tests.} \end{aligned}$$

The key interpretive caveat is that Lane B is a search process, not a production sample. Its missing-mass estimates should therefore be interpreted as saturation signals for the *targeted search frontier*, not as direct estimates of production traffic risk.

4 Method

4.1 Representative-lane rate bound

For each slice h , we compute a one-sided exact upper confidence limit:

$$U_h(\alpha) = \text{Beta}^{-1}(1 - \alpha; x_h + 1, n_h - x_h), \quad x_h < n_h, \quad (1)$$

where Beta^{-1} is the inverse cumulative distribution function of the beta distribution. For $x_h = 0$ this simplifies to

$$U_h(\alpha) = 1 - \alpha^{1/n_h}. \quad (2)$$

At 95% confidence, $U_h(0.05) \approx 3/n_h$ when $x_h = 0$ [Hanley and Lippman-Hand, 1983]. The zero-failure sample-size requirement for threshold τ_h is therefore

$$n_h \geq \frac{\log(\alpha)}{\log(1 - \tau_h)} \approx \frac{-\log(\alpha)}{\tau_h}. \quad (3)$$

At 95% confidence, roughly 300 zero-failure opportunities are needed to bound the severe-failure rate below 1%, 600 for 0.5%, and 3000 for 0.1%.

4.2 Representative-lane rate bound

For each slice h , we compute a one-sided exact upper confidence limit:

$$U_h(\alpha) = \text{Beta}^{-1}(1 - \alpha; x_h + 1, n_h - x_h), \quad x_h < n_h, \quad (4)$$

where Beta^{-1} is the inverse cumulative distribution function of the beta distribution. For $x_h = 0$ this simplifies to

$$U_h(\alpha) = 1 - \alpha^{1/n_h}. \quad (5)$$

At 95% confidence, $U_h(0.05) \approx 3/n_h$ when $x_h = 0$ [Hanley and Lippman-Hand, 1983]. The zero-failure sample-size requirement for threshold τ_h is therefore

$$n_h \geq \frac{\log(\alpha)}{\log(1 - \tau_h)} \approx \frac{-\log(\alpha)}{\tau_h}. \quad (6)$$

At 95% confidence, roughly 300 zero-failure opportunities are needed to bound the severe-failure rate below 1%, 600 for 0.5%, and 3000 for 0.1%.

When $x_h > 0$, the same exact upper bound in Eq. (1) applies, but the zero-failure approximation no longer holds. In this case, release readiness is determined directly by whether

$$U_h(\alpha) \leq \tau_h. \quad (7)$$

Equivalently, for fixed (α, τ_h, x_h) , one may solve numerically for the smallest n_h such that

$$\text{Beta}^{-1}(1 - \alpha; x_h + 1, n_h - x_h) \leq \tau_h. \quad (8)$$

This makes clear that observing even a small number of uncaught severe failures can substantially increase the evidence required to justify release at a given threshold. In practice, nonzero-failure slices will often be better handled by fixing the release candidate and re-evaluating, rather than attempting to “test through” the failures with a much larger sample.

4.3 Targeted-lane saturation statistics

The targeted lane tracks whether the search frontier is flattening.

Discovery curve. The curve K_t reveals whether cumulative unique severe mechanisms are still growing materially late in testing.

Rolling novelty. The quantity $\Delta K_t(w)$ measures how many severe types were first observed in the most recent window of w targeted evaluations. If $\Delta K_t(w) = 0$ for repeated windows, the search process has locally flattened.

Good-Turing missing mass. Let $N_{\text{err}}(t)$ be the number of severe-error observations up to time t , and let $f_1(t)$ be the number of singleton severe types. The Good-Turing missing-mass estimate is

$$M_t = \frac{f_1(t)}{N_{\text{err}}(t)}. \quad (9)$$

High singleton fractions imply a jagged, incompletely explored tail; low singleton fractions imply increasing saturation [Good, 1953].

Optional richness lower bound. When desired, one may also compute a Chao lower-bound richness estimate

$$S_{\text{Chao}}(t) = S_{\text{obs}}(t) + \frac{f_1(t)^2}{2f_2(t)}, \quad (10)$$

for $f_2(t) > 0$, with the usual bias-corrected fallback when $f_2(t) = 0$ [Chao and Jost, 2012]. This yields a completeness proxy $C_t = S_{\text{obs}}(t)/S_{\text{Chao}}(t)$.

4.4 Composite stopping rule

Let α be the representative-lane confidence level, τ_h the per-slice severe-failure threshold, ρ the maximum allowed missing mass under the targeted search process, and w the rolling novelty window. We stop only when all four conditions hold:

C1. Rate bound: $U_h(\alpha) \leq \tau_h$ for every deployment-critical slice h .

C2. Recent saturation: $\Delta K_t(w) = 0$ for the current targeted window.

C3. Tail mass: $M_t \leq \rho$ after a minimum number of severe observations have been accumulated.

C4. Operational readiness: the release candidate is instrumented for monitoring, escalation, rollback, and post-deployment TEVV.

Condition C4 is not a statistical criterion, but it matters for deployment readiness because stopping is not a claim of perfection. It is a claim that the remaining risk is bounded and operationally manageable [National Institute of Standards and Technology, 2023, Bloomfield and Rushby, 2024].

The proposed rule does not claim to estimate the entire space of future failures, and it is not a universal threshold schedule. Instead, it generates a narrow and reviewable claim about deployment. It says that given the release candidate, severity taxonomy, slices, and targeted search, additional testing is unlikely to materially change the release decision because the failure rate is bounded below threshold and the targeted discovery frontier has flattened.

5 Monte Carlo study

5.1 Design

We evaluated the proposed rule in a stylized Monte Carlo study designed to test two failure modes of naive stopping rules: (i) stopping when representative risk is still too high, and (ii) stopping when long-tail novel severe types are still materially undiscovered.

Each replication contains four representative slices and one targeted lane. Representative-lane observations are independent Bernoulli trials with slice-specific post-control severe-failure probabilities p_h . The targeted lane is a category-discovery process: each targeted evaluation triggers a severe event with probability $q_{\text{target}} = 0.25$, and conditional on a severe event, the error type is sampled from a Zipf distribution over J latent mechanisms with exponent s . Larger J and smaller s induce a longer tail.

The simulation uses $\alpha = 0.05$, $\tau_h = 1\%$ for all slices, $\rho = 10\%$, a rolling novelty window of $w = 100$ targeted tests, 25 representative evaluations per slice per round, 25 targeted evaluations per round, and a maximum of 60 rounds. Thus the maximum budget is 1500 representative opportunities per slice and 1500 targeted evaluations. We ran 300 replications per scenario.

5.2 Scenarios

Table 1 summarizes the four scenario families. The safe scenarios differ only in the tail of latent severe mechanisms. The near-threshold scenario is intentionally borderline. The unsafe scenario has maximum slice risk above the release threshold.

Table 1: Simulation scenarios. The severe-failure threshold is $\tau = 1\%$.

Scenario	$\max_h p_h$	$\max_h p_h / \tau$	J	Zipf exponent s
Safe, short tail	0.40%	0.40	12	1.4
Safe, long tail	0.40%	0.40	60	0.8
Near-threshold, long tail	0.80%	0.80	60	0.8
Unsafe, long tail	1.50%	1.50	60	0.8

5.3 Baseline rules

We compare the proposed method against three simpler baselines:

- (a) **Fixed budget:** stop after 300 representative opportunities per slice and 300 targeted evaluations.
- (b) **Recent novelty:** stop once no new severe type has appeared in the last 100 targeted evaluations, provided at least 300 representative opportunities per slice have been collected.
- (c) **Rate only:** stop once the representative-lane rate bound is satisfied, ignoring tail saturation.

5.4 Evaluation metrics

For each run we record:

- **Premature-stop rate:** the rule stopped even though either $\max_h p_h > \tau$ or the true undiscovered severe-type mass under the targeted search distribution exceeded ρ at the stop time.
- **Budget consumed:** total evaluations used before stopping.
- **Missed novelty:** the number of previously unseen severe types discovered in the next 200 targeted evaluations after the rule stopped.

5.5 Results

Figure 1 shows the most important result. Fixed-budget stopping performs poorly whenever the tail is long: it stops prematurely in 100% of safe-long-tail, near-threshold-long-tail, and unsafe-long-tail replications. Recent-novelty stopping is less brittle than a fixed budget, but it still stops prematurely 22.7% of the time in the safe-long-tail scenario and 100% of the time in the unsafe-long-tail scenario because the novelty signal can flatten even while representative risk remains unacceptable.

The comparison between the rate-only and dual rules is the critical one. In the safe short-tail scenario, the two rules behave nearly identically: both have 0% premature stops and discover

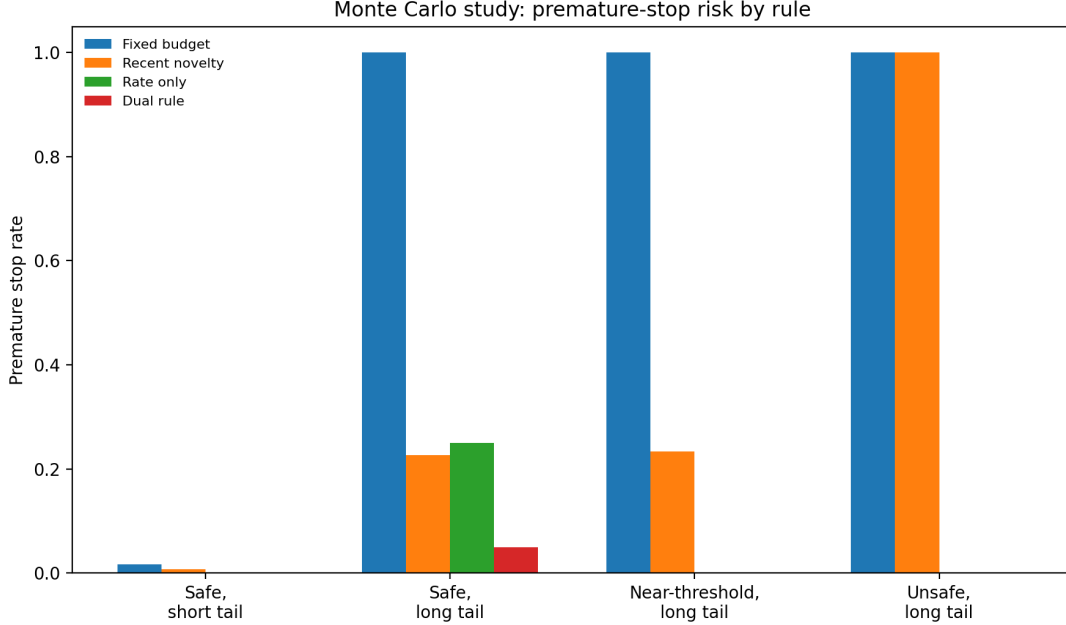


Figure 1: Premature-stop risk across the Monte Carlo scenarios. A stop is premature if representative risk is above threshold or the true undiscovered severe-type mass in the targeted search process still exceeds $\rho = 10\%$.

almost no additional severe types in the next 200 targeted tests after stopping. The extra saturation machinery is therefore not costly when the tail is already simple. In the safe long-tail scenario, however, the difference becomes visible. The rate-only rule stops in 90.0% of runs after an average of 4630.8 evaluations, but 25.0% of those stops are premature and an average of 2.86 novel severe types are still found in the next 200 targeted tests. The dual rule stops less often (76.7%) and later (5862.1 evaluations), but premature stops fall to 5.0% and post-stop novel severe types fall to 1.59 on average.

In the unsafe long-tail scenario, both the rate-only and dual rules refuse to stop within the available budget, which is conservative but desirable. By contrast, the recent-novelty rule stops in every run and is wrong every time. The near-threshold scenario behaves similarly: the dual rule almost never stops within budget. This is not a bug in the method; it is a signal that a borderline system has not accumulated enough evidence to justify release at the chosen threshold.

Figure 2 reinforces the same point from another angle. Fixed budgets miss many severe types in long-tail settings, while the dual rule substantially reduces what remains to be discovered after the stop point. Table 2 reports the full numerical results.

5.6 Takeaways from the simulation

Three lessons stand out. First, a fixed budget is not a stopping rule; it is a budget cap. It can be acceptable for planning, but not as a sufficiency argument. Second, recent novelty alone is not enough because targeted discovery can plateau even while representative risk remains too high. Third, rate bounds alone can still release too early in long-tail settings because they do not ask whether the search frontier has flattened. The dual rule therefore buys something real: lower premature-stop risk in exactly the settings where tail novelty matters.

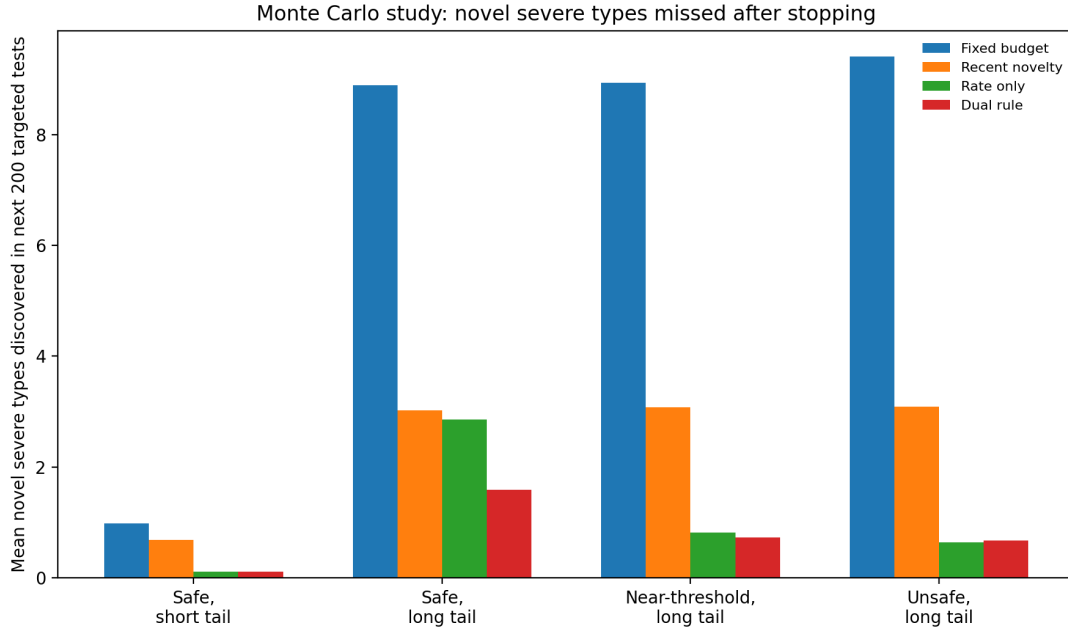


Figure 2: Mean number of previously unseen severe types discovered in the next 200 targeted evaluations after stopping. Lower is better.

Table 2: Monte Carlo operating characteristics (300 replications per scenario).

Scenario	Rule	Mean evals	Stop rate	Premature stop	Mean missed novel
Safe, short tail	Fixed budget	1500.0	1.000	0.017	0.98
Safe, short tail	Recent novelty	1793.3	1.000	0.007	0.68
Safe, short tail	Rate only	4674.2	0.893	0.000	0.11
Safe, short tail	Dual rule	4679.6	0.893	0.000	0.12
Safe, long tail	Fixed budget	1500.0	1.000	1.000	8.89
Safe, long tail	Recent novelty	4224.6	1.000	0.227	3.03
Safe, long tail	Rate only	4630.8	0.900	0.250	2.86
Safe, long tail	Dual rule	5862.1	0.767	0.050	1.59
Near-threshold, long tail	Fixed budget	1500.0	1.000	1.000	8.93
Near-threshold, long tail	Recent novelty	4205.4	1.000	0.233	3.08
Near-threshold, long tail	Rate only	7428.8	0.040	0.000	0.82
Near-threshold, long tail	Dual rule	7462.9	0.020	0.000	0.73
Unsafe, long tail	Fixed budget	1500.0	1.000	1.000	9.41
Unsafe, long tail	Recent novelty	4249.2	1.000	1.000	3.09
Unsafe, long tail	Rate only	7500.0	0.000	0.000	0.65
Unsafe, long tail	Dual rule	7500.0	0.000	0.000	0.68

6 Illustrative case study

To demonstrate how the method looks in practice, we include a synthetic evaluation log for a policy-oriented retrieval assistant. The data are not intended as empirical validation of the framework; the simulation above plays that role. The case study is instead an end-to-end worked example of the evidence record that a release team could produce.

The representative lane contains four critical slices: common policy questions, exception handling, multi-document conflict resolution, and abstain-or-escalate behavior. The targeted lane contains 280 adversarial or tail-focused evaluations. Figure 3 shows the representative-lane rate bound, and Figure 4 shows the targeted-lane discovery curve.

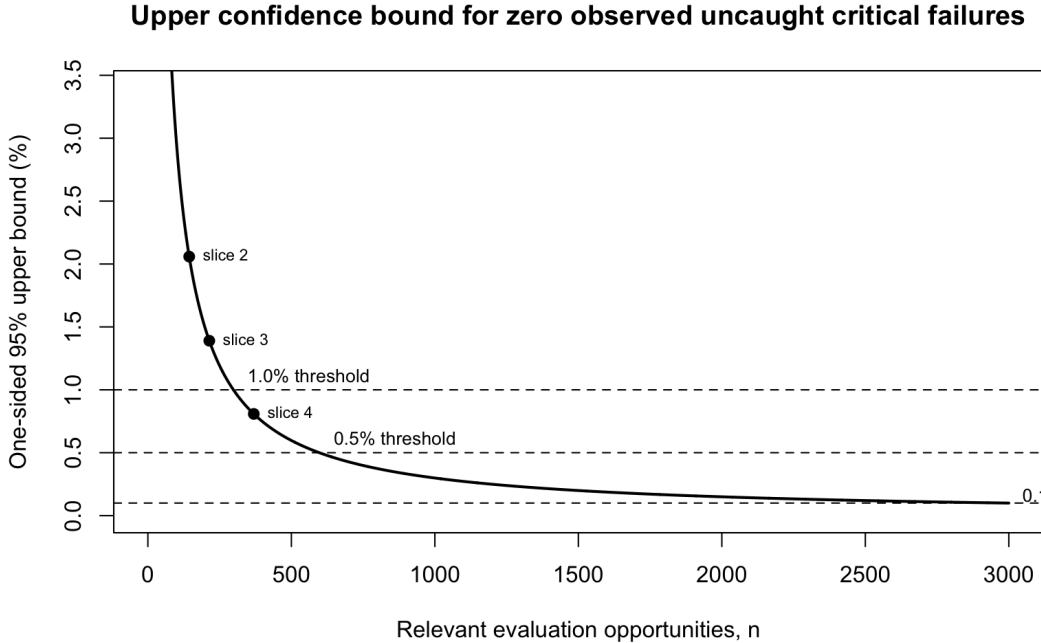


Figure 3: Representative-lane upper confidence bound as a function of sample size, with the synthetic case-study slice totals overlaid. All four slices observed zero uncaught critical failures.

Table 3 shows the representative-lane summary. Each slice observed zero uncaught critical failures, yielding one-sided 95% upper bounds between 0.46% and 0.50%. This would satisfy a 0.5% critical-risk gate. In the targeted lane, 24 severe events were observed across 280 evaluations, corresponding to 7 distinct severe mechanisms. The final 100 targeted evaluations did not introduce new severe type; the Good-Turing missing-mass estimate was 4.17%; and a Chao-style completeness estimate was 96.6%. Under a policy threshold of $\rho = 10\%$, the synthetic case therefore satisfies both the representative and saturation conditions.

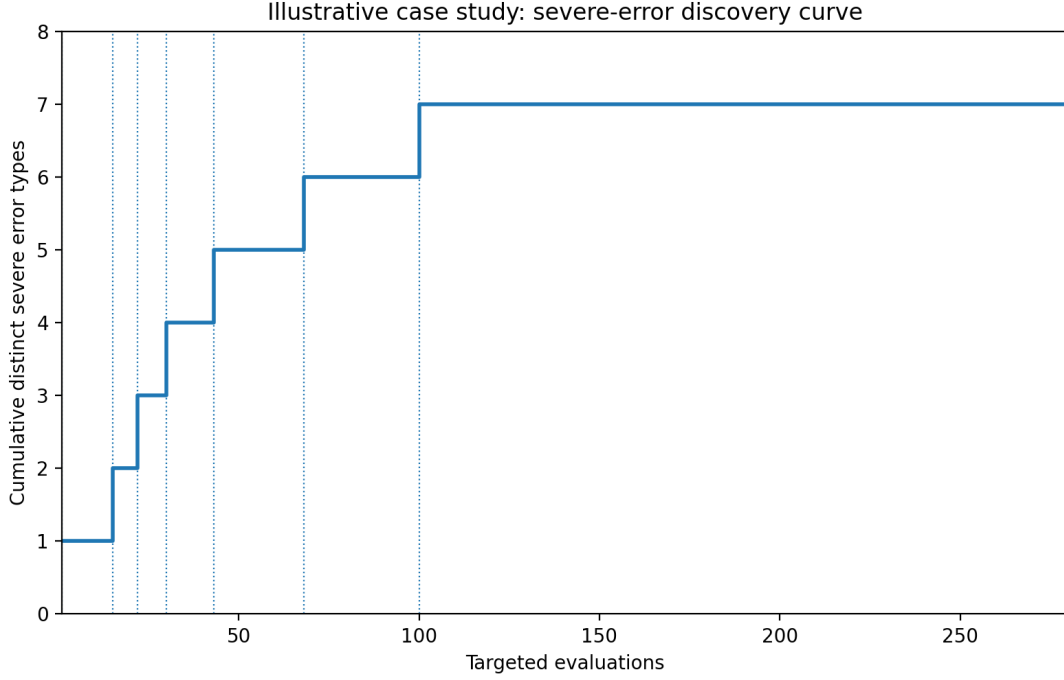


Figure 4: Case-study targeted discovery curve. Seven distinct severe error mechanisms are found, with no new severe type discovered in the final 100 targeted evaluations.

Table 3: Representative-lane summary for the synthetic case study.

Slice	Relevant opportunities	Uncaught critical	Upper 95% bound
Abstain / escalate	600	0	0.498%
Common policy	650	0	0.460%
Exception handling	620	0	0.482%
Multi-document conflict	610	0	0.490%

7 Discussion

7.1 What this method does and does not claim

The method is not a proof of safety, correctness, or completeness. It does not guarantee that no harmful failure will occur after deployment. Nor does it produce universal thresholds that should apply across all organizations or use cases. Its claim is more limited and therefore more defensible: for a specified release candidate and search process, the method makes explicit why continued evaluation is or is not materially reducing uncertainty.

This distinction matters because much practical disagreement about AI release readiness is not really disagreement about whether testing is important. It is disagreement about what a stop decision should mean. The present paper takes a middle position between “ship after a benchmark” and “test forever.” It treats the stop decision as a bounded assurance claim.

7.2 The role of Lane B

The targeted lane is the least standard part of the framework and also the most important. It is easy to say that LLM behavior has a long tail. It is harder to incorporate that fact into a release gate. The proposed use of discovery curves, rolling novelty, and missing mass is one way to do so. The interpretation must remain careful. Missing mass in the targeted lane does *not* estimate production failure probability. Rather, it estimates how much of the *search-distribution* severe-type mass appears to remain unseen. In practice, that is still useful. Release teams are often not asking, “What is the exact production probability of mechanism type 37?” They are asking, “Are we still learning important new ways the system breaks?” Lane B addresses that question directly.

7.3 Why the simulation matters

Without the simulation section, the framework could be dismissed as an appealing governance story. The Monte Carlo results move it closer to a methods paper. They show that the dual rule has interpretable operating characteristics, that it is not uniformly better on every dimension, and that its main value appears in long-tail scenarios where simple heuristics are fragile. That is the correct way to position the framework academically: not as a universal theorem, but as a tunable sequential method with measurable trade-offs.

7.4 From paper to organizational adoption

Although the paper is framed as a methods contribution, its outputs map naturally to organizational processes. The stopping rule yields the raw material for a short release memo or assurance case: the per-slice rate claim, the tail-saturation claim, and the operational-control claim. In that sense, the framework provides a bridge between statistical evidence and practical deployment governance.

8 Limitations

Several limitations remain.

First, the simulation is stylized. Targeted discovery is modeled as a Zipfian category process, which is useful for stress-testing the rule but is not a literal model of human red-team behavior or adaptive

agentic interactions. More realistic simulations could incorporate clustered prompts, nonstationarity across model revisions, tool-mediated multi-step episodes, and adversarial adaptation.

Second, the paper focuses on severe failures and does not attempt a full multi-objective release policy. Real deployment decisions trade off quality, latency, cost, user value, and multiple harm types simultaneously. Future work could embed the stopping rule in a broader decision-theoretic framework in the spirit of classic stop-testing research [Dalal and Mallows, 1988].

Third, the taxonomy of “distinct severe types” is partly judgment-based. Poor taxonomies can make novelty appear to saturate too early or never saturate at all. This makes taxonomy design a first-class part of the method.

Fourth, the current rule assumes a fixed release candidate. In real development, teams fix issues while testing. Sequential rules under changing system configurations are an important next step.

9 Conclusion

Pre-deployment AI evaluation needs a stopping rule that is more principled than “we tested a lot” and less absolute than “we proved the system safe.” This paper proposes one such rule. Deployment readiness should require both a representative-lane rate bound and a targeted-lane saturation claim. The first bounds how often severe failures occur in the slices that matter. The second asks whether targeted testing is still discovering materially new severe mechanisms. Together they produce a transparent and reviewable argument for release sufficiency.

The Monte Carlo study suggests that this dual rule is most valuable in long-tail settings, where fixed budgets and novelty-only heuristics can stop too early and rate-only rules can still miss important undiscovered mechanisms. The synthetic case study shows how the evidence can be recorded in practice. For organizations building deployment frameworks or centers of excellence, the immediate benefit is not a universal number of required tests. It is a defensible answer to a harder and more realistic question: *why was stopping justified here?*

References

- Rasmus Adler, Michael Klaes, et al. Assurance cases as foundation stone for auditing ai-enabled and autonomous systems: Workshop results and political recommendations for action from the examai project. *arXiv preprint arXiv:2208.08198*, 2022. doi: 10.48550/arXiv.2208.08198.
- Robin Bloomfield and John Rushby. Assurance of ai systems from a dependability perspective. *arXiv preprint arXiv:2407.13948*, 2024.
- Anne Chao and Lou Jost. Coverage-based rarefaction and extrapolation: Standardizing samples by completeness rather than size. *Ecology*, 93(12):2533–2547, 2012. doi: 10.1890/11-1952.1.
- C. J. Clopper and E. S. Pearson. The use of confidence or fiducial limits illustrated in the case of the binomial. *Biometrika*, 26(4):404–413, 1934. doi: 10.1093/biomet/26.4.404.
- S. R. Dalal and C. L. Mallows. When should one stop testing software? *Journal of the American Statistical Association*, 83(403):872–879, 1988. doi: 10.1080/01621459.1988.10478676.
- Amrit L. Goel and Kazuhira Okumoto. Time-dependent error-detection rate model for software reliability and other performance measures. *IEEE Transactions on Reliability*, R-28(3):206–211, 1979.
- I. J. Good. The population frequencies of species and the estimation of population parameters. *Biometrika*, 40(3/4):237–264, 1953. doi: 10.1093/biomet/40.3-4.237.
- James A. Hanley and Abby Lippman-Hand. If nothing goes wrong, is everything all right? interpreting zero numerators. *JAMA*, 249(13):1743–1745, 1983. doi: 10.1001/jama.1983.03330370053031.

- Tim Kelly and Rob Weaver. The goal structuring notation—a safety argument notation. In *Proceedings of the Workshop on Assurance Cases*, 2004.
- Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, et al. Holistic evaluation of language models. *Transactions on Machine Learning Research*, 2023.
- Margaret Mitchell, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. Model cards for model reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency*, pages 220–229, 2019. doi: 10.1145/3287560.3287596.
- John D. Musa and Kazuhira Okumoto. A logarithmic poisson execution time model for software reliability measurement. In *Proceedings of the 7th International Conference on Software Engineering*, pages 230–238, 1984.
- National Institute of Standards and Technology. Artificial intelligence risk management framework (AI RMF 1.0). Technical Report NIST AI 100-1, National Institute of Standards and Technology, 2023.
- National Institute of Standards and Technology. Artificial intelligence risk management framework: Generative artificial intelligence profile. Technical Report NIST AI 600-1, National Institute of Standards and Technology, 2024.